Inventory Intelligence · Project Deep Dive

What Is It?

Inventory Intelligence is a full-stack demand planning platform I built for DTC (Direct-to-Consumer) e-commerce brands. It connects to Shopify and ShipHero, pulls in real order and inventory data, then uses statistical forecasting to predict demand and automate purchase order decisions.

Think of it as a clone of tools like Inventory Planner or Prediko · the kind of software a brand like Otishi (a DTC fitness/lifting company) would use to stop running out of their bestselling shoe sizes while avoiding excess stock on slow movers.

Tech Stack

| Layer | Technology | |-------|-----------| | **Backend** | Python, FastAPI (async), SQLAlchemy 2.0 (async ORM) | | **Forecasting** | Pandas, NumPy, Statsmodels (Holt-Winters exponential smoothing) | | **Frontend** | React 18, Vite, Recharts, React Router, Axios | | **Database** | SQLite (dev), PostgreSQL (prod via Docker) | | **Scheduling** | APScheduler (cron + interval triggers) | | **Integrations** | Shopify Admin REST API, ShipHero GraphQL API | | **Deployment** | Docker Compose (PostgreSQL + FastAPI + Vite) |

Architecture Overview

```
...............      ...........................................
·   Shopify    ······>·              FastAPI Backend              ·
·  REST API    ·      ·                                          ·
...............      ·  Routers (7):                            ·
                      ·    dashboard, inventory, forecasting,    ·
...............      ·    analytics, purchase_orders,          ·
·  ShipHero   ······>·    promotions, sync                     ·
· GraphQL API ·      ·                                          ·
...............      ·  Services:                               ·
                      ·    forecasting (Holt-Winters engine)     ·
                      ·    analytics (ABC, dead/excess stock)    ·
                      ·    reconciliation (cross-source compare) ·
                      ·    shopify_sync / shiphero_sync          ·
                      ·    scheduler (APScheduler · 4 cron jobs) ·
                      ·                                          ·
                      ·  Database: SQLAlchemy async              ·
                      ·    7 models, dialect-aware upserts       ·
                      ...........................................
                                     ·
                      ...........................................
                      ·          React 18 Frontend               ·
                      ·  Dashboard, Inventory, Forecasting,      ·
                      ·  Reports (ABC/Dead/Excess), PO Generator, ·
                      ·  Promotions, Setup/Connection Testing    ·
                      ...........................................
```

Key Features & How I Built Them

1. Demand Forecasting (Holt-Winters Exponential Smoothing)

This is the core of the app. For each SKU, the system:

1. **Pulls 365 days of daily sales** from the database (aggregated from order line items)
2. **Fills gaps** · days with zero sales get explicit zeros so the time series is continuous
3. **Removes outliers** using z-score filtering (threshold = 3.0) · this protects against one-off bulk orders skewing the forecast
4. **Detects seasonality** via autocorrelation at lag-7 (weekly) and lag-30 (monthly). If either autocorrelation > 0.3, the SKU is flagged as seasonal
5. **Fits a Holt-Winters model** · additive trend + additive seasonality (7-day period) if seasonal, or additive trend only if not
6. **Forecasts 30/60/90 days** ahead and computes reorder points, days of stock remaining, and suggested order quantities

Why Holt-Winters over a simple moving average? Moving averages can't capture trend direction or repeating seasonal patterns. A shoe brand sees predictable spikes in January (New Year resolutions) and November (Black Friday). Holt-Winters decomposes the signal into level + trend + seasonal components, so it naturally handles these patterns.

Fallback logic: If a SKU has fewer than 14 days of data, or the model fit fails, it falls back to mean-based forecasting. The system is designed to never crash on bad data.

---

2. ABC Classification (Pareto Analysis)

Classifies every SKU by revenue contribution using the Pareto principle:

- **Class A** (top 80% of revenue) · your critical few
- **Class B** (next 15%) · solid contributors
- **Class C** (bottom 5%) · long tail

This drives smarter decisions: Class A SKUs get aggressive reorder rules and tighter monitoring, while Class C SKUs might be candidates for discontinuation.

---

3. Dead Stock & Excess Stock Detection

Dead Stock: Identifies SKUs with inventory sitting in the warehouse but no recent sales. Gives actionable recommendations based on severity ·"Liquidate or write off" for 180+ days idle, "Run promotion" for 60-90 days, " Monitor" for borderline cases.

Excess Stock: Calculates how many units exceed your target coverage window (e.g., 90 days of supply). Shows the dollar value tied up in excess inventory and gives ABC-aware recommendations · a slow-moving Class C item with 400 days of stock gets "Aggressive markdown," while a Class A item just gets "Pause reorders."

---

4. Inventory Reconciliation

Joins Shopify and ShipHero inventory snapshots by SKU to flag discrepancies. ShipHero is treated as the source of truth (it's the warehouse system), and any variance against Shopify is surfaced so the brand can investigate.

---

5. Purchase Order Suggestions

Auto-generates purchase orders ranked by urgency:

- **Critical:** Days of stock remaining <= lead time (you'll stock out before the order arrives)
- **High:** <= 1.5x lead time
- **Medium:** <= 2x lead time
- **Low:** Comfortable runway

Each suggestion includes the SKU, current stock, reorder point, suggested quantity (8 weeks of supply minus current stock), daily velocity, and lead time. Exportable to CSV.

---

6. API Integrations

Shopify (REST API):

- Paginated product/order/inventory fetches with Link header parsing
- Rate limiting: monitors `X-Shopify-Shop-Api-Call-Limit` header, backs off when approaching the 40-request bucket limit
- 5 retries with exponential backoff, respects `Retry-After` headers

ShipHero (GraphQL API):

- Cursor-based pagination using `pageInfo.endCursor`
- Throttle detection in three places: HTTP 429 status, `Retry-After` header, and GraphQL response body (complexity field)
- Same 5-retry exponential backoff strategy

Engineering Decisions Worth Discussing

Batch Queries & N+1 Prevention

The forecasting engine needs sales data, product titles, reorder rules, and current stock for every SKU. A naive approach would be 4 queries per SKU (4N total). Instead, I use 4 batch queries that fetch everything at once, then group in-memory with Pandas. For 60 SKUs, that's 4 queries instead of 240.

30-Minute Forecast Cache

Forecast computation is expensive (Holt-Winters model fitting for every SKU). I cache the results in-memory with a 30-minute TTL. The cache is invalidated nightly when the scheduler recalculates. Trade-off: data can be up to 30 minutes stale after a sync, but page loads are instant.

Dialect-Aware Upserts

The app runs on SQLite in development and PostgreSQL in production. Both support "upsert" (INSERT ... ON CONFLICT DO UPDATE), but the syntax differs. I wrote a `get_upsert()` helper that detects the dialect and returns the correct function, so the same code works in both environments.

Immutable Inventory Snapshots

Instead of updating a single row when stock levels change, I insert a new `InventorySnapshot` record on every sync. This creates a time series of stock levels that could be used for historical analysis. I query the "latest per SKU" using a `MAX(id)` subquery grouped by SKU.

No Migrations in Dev

Instead of Alembic migrations during development, the app uses `create_all()` on startup plus a `_add_missing_columns()` helper that introspects the database and runs `ALTER TABLE ADD COLUMN` for any model fields that don't exist yet. This keeps iteration fast. In production, you'd use proper migrations.

Outlier Removal Before Forecasting

A single bulk order (say, 500 units when the daily average is 5) would massively skew a forecast. The z-score filter catches these: any daily value more than 3 standard deviations from the mean gets replaced with the mean. This makes forecasts robust against anomalous data.

Seed Data & Demo

The seed script generates 60 realistic SKUs modeled after Otishi.com's product catalog:

- **44 shoe variants** (4 colorways x 11 sizes) with realistic size-curve demand
- **10 tee variants**, **10 sweatshirt variants**, socks, lifting straps, shaker bottles
- **365 days of order history** with seasonal multipliers (Jan: 1.6x for New Year, Nov: 1.5x for Black Friday, Dec: 1.7x for holidays)
- **~8,000 order line items** with weighted SKU selection (bestsellers sell more)
- **Two warehouses** (NJ and CA) with realistic stock conditions including some stockouts, low stock, and excess inventory
- **Shopify vs. ShipHero variance** (±1-3 units) to demonstrate reconciliation

---

Frontend Highlights

- **Dashboard** with 10 KPI cards, sales trend chart, reorder alerts, and data freshness indicator
- **Sortable, paginated tables** across every page using shared components (`SortableHeader`, `Pagination`, `tableStyles`)
- **InfoTooltips** on every metric explaining what it means and why it matters
- **Card view toggle** for visual inventory browsing with product images
- **Stock drawdown chart** per SKU showing projected runway at current velocity
- **CSV export** on all report tabs (ABC, dead stock, excess stock, PO suggestions)
- **Connection testing UI** for Shopify and ShipHero with step-by-step setup guide
- **Consistent design system** using CSS variables · no external CSS framework

---

Database Schema (7 Tables)

Table	Purpose	Key Fields
Product	SKU catalog	sku (unique), title, variant_id, shopify_id, shiphero_id, image_url, cost_price, parent_sku
Order	Order headers	order_id, shopify_order_id, created_at, total_price
OrderLineItem	Order details	line_item_id, order_id (FK), sku, quantity, price
InventorySnapshot	Stock levels (immutable)	sku, quantity_on_hand/allocated/available, warehouse, source, recorded_at
ReorderRule	Per-SKU reorder config	sku, reorder_point, reorder_quantity, lead_time_days, safety_stock
PromotionalPeriod	Promo tracking	name, start_date, end_date, discount_pct, notes
JobLog	Sync/job history	job_name, status, started_at, finished_at, records_processed, error_message

---

Background Job Schedule

Job	Trigger	What It Does
Shopify Sync	Cron: 2am UTC	Full sync of products, orders (365d), inventory levels
Forecasting	Cron: 3am UTC	Recalculate all forecasts, update reorder rules
ShipHero Sync	Every 4 hours	Sync inventory, fetch open POs
Reconciliation	Every 4 hours (offset 30min)	Compare Shopify vs ShipHero, flag discrepancies

All jobs log to the `JobLog` table with status, timing, and record counts. Manual triggers available via the UI.

What I'd Add Next

- **ML-based forecasting** (XGBoost) trained on historical accuracy to improve on Holt-Winters
- **Promotional period integration** into the forecasting engine (adjust predictions during known sales events)
- **Multi-tenant support** for serving multiple brands from one deployment
- **Webhook listeners** for real-time Shopify order events instead of polling
- **Historical stock level charts** leveraging the immutable snapshot architecture